

Implementasi Shamir Secret Sharing pada Aplikasi Password Manager Terdistribusi



Disusun oleh:
Kelompok 6

Raja M Arkan

10123065

Muhammad Arya Putra
Prihastono

18223068

Maheesa Satria Prayata

18223082

**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026**

DAFTAR ISI

DAFTAR ISI.....	2
DAFTAR TABEL.....	4
DAFTER GAMBAR.....	5
PERNYATAAN AI.....	7
Teori Singkat.....	8
SSS (Shamir Secret Sharing).....	8
AES-GCM.....	10
KDF (Key Derivation Function).....	12
CSPRNG (Cryptographically Secure Pseudorandom Number Generator).....	14
Kriptografi Visual.....	16
Zero Knowledge Proof.....	17
Perancangan dan Implementasi.....	19
Arsitektur Umum.....	19
Infrastruktur Teknologi.....	20
Struktur Direktori.....	21
KDF Argon2id.....	22
AES-128-GCM.....	23
Shamir Secret Sharing.....	24
CSPRNG Password Generator.....	24
Share Format.....	25
Alur Pembuatan Vault.....	25
Session State Management.....	26
Penyimpanan Lokal.....	28
Server Zero Knowledge.....	28
Antarmuka Pnegguna (CLI).....	30
Bonus - Kriptografi Visual.....	30
Pengujian Program dan Analisis.....	32
Uji Pembuatan Vault.....	32
Uji penyimpanan dan perlindungan local share.....	35
Uji akses normal.....	37
Uji Penambahan Data Password.....	40

Uji Pengubahan dan Penghapusan Data Password.....	42
Uji Penyimpanan Vault di Server.....	46
Uji Mode Backup.....	48
Uji Kegagalan Pemulihan.....	50
Uji Kriptografi Visual bonus.....	53
Kesimpulan dari Hasil Implementasi.....	56
Daftar Pustaka.....	57
Lampiran.....	58

DAFTAR TABEL

Tabel 1.1 Sifat-sifat Shamir Secret Sharing (SSS).....	9
Tabel 1.2 Komponen Input dan Output AES-GCM.....	11
Tabel 1.3 Parameter Argon2.....	13
Tabel 1.4 Perbandingan Random Generator.....	15
Tabel 1.5 Sifat Fundamental Zero Knowledge Proof.....	17
Tabel 2.1 Tempat Penyimpanan Share.....	19
Tabel 2.2 Fungsi, Teknologi/Library, dan Alasan Pemilihan Teknologi.....	20
Tabel 2.3 Struktur Direktori dan Tanggung Jawabnya Masing-Masing.....	21
Tabel 2.4 Parameter Argon2id.....	22
Tabel 2.5 Lokasi Setiap Share Setelah pembuatan Vault.....	26
Tabel 2.6 File yang disimpan di Lokal.....	28
Tabel 2.7 Dokumentasi endpoint.....	29
Tabel 2.8 Implementasi Kriptografi Visual.....	30

DAFTAR GAMBAR

Gambar 1.1 Skema Shamir Secret Sharing (SSS).....	8
Gambar 1.2 Skema Argon2.....	12
Gambar 1.3 Kriptografi Visual Skema (2,2).....	15
Gambar 1.4 Skema Zero Knowledge Proofs.....	16
Gambar 2.1 Flowchart Pembuatan Vault.....	24
Gambar 2.2 Flowchart Backup dan Normal Login.....	25
Gambar 3.1 Pembuatan vault baru.....	30
Gambar 3.2 Recovery Share.....	31
Gambar 3.3 Server Share.....	31
Gambar 3.4 Local Share.....	32
Gambar 3.5 Recovery Share Memuat Koordinat Share dan Nilai Share.	32
Gambar 3.6 Data yang Disimpan Server.....	33
Gambar 3.7 Local Share disimpan dalam bentuk base 64.....	33
Gambar 3.8 Local Share Berhasil Dienkripsi.....	34
Gambar 3.9 Local Share Berhasil Didekripsi.....	34
Gambar 3.10 Local Share Gagal Didekripsi.....	35
Gambar 3.11 Local Share Digunakan.....	36
Gambar 3.12 Server Share Dipakai Dengan Local Share.....	36
Gambar 3.13 Isi Vault Kosong Dapat Dilihat.....	37
Gambar 3.14 Vault Tidak Dapat Diakses.....	37
Gambar 3.15 Penambahan Data Password.....	38
Gambar 3.16 Penambahan Data Password Dengan CSPRNG.....	39
Gambar 3.17 Enkripsi Ulang Vault Dengan Nonce Baru.....	39
Gambar 3.18 Backup Vault Sebelumnya.....	40
Gambar 3.19 Backup Vault Setelah.....	40
Gambar 3.20 Password diubah setelah diedit.....	41
Gambar 3.21 Vault Berhasil Dienkripsi Ulang.....	41
Gambar 3.22 Data Password Berhasil Dihapus.....	42
Gambar 3.23 Vault Berhasil Dienkripsi Ulang.....	42
Gambar 3.24 Menu pada Mode Normal.....	43
Gambar 3.25 Menu pada Mode Backup.....	43

Gambar 3.26 Data Vault disimpan dalam bentuk BLOB.....	44
Gambar 3.27 Data Vault disimpan terenkripsi.....	44
Gambar 3.28 Test Script yang Dipakai.....	45
Gambar 3.29 Hasil Pengujian Test Script.....	45
Gambar 3.30 Server Dimatikan.....	46
Gambar 3.31 Backup vault dibuka.....	46
Gambar 3.32 Kombinasi Local Share dengan Backup Share.....	47
Gambar 3.33 Backup Vault yang diakses.....	47
Gambar 3.34 Data Password diperlihatkan pada Mode Backup.....	47
Gambar 3.35 Opsi menambah, mengubah, atau menghapus data tidak tersedia.....	48
Gambar 3.36 Dekripsi Backup Vault Gagal karena Recovery Share yang Salah.....	49
Gambar 3.37 Kondisi Awal Vault.....	50
Gambar 3.38 Kondisi Vault Dengan Nonce Diubah.....	50
Gambar 3.39 Vault Backup Tidak dapat diakses.....	51
Gambar 3.40 QR code Recovery Share Awal.....	51
Gambar 3.41 Share 1.....	52
Gambar 3.42 Share 2.....	52
Gambar 3.43 Hasil penggabungan share.....	53
Gambar 3.44 Hasil Scan QR rekonstruksi.....	53
Gambar 3.45 Script String checker.....	54
Gambar 3.46 Hasil Pengecekan String.....	54

PERNYATAAN AI

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (raw output) dari Generative AI, melainkan hasil pengembangan dan penulisan mandiri.

A stylized handwritten signature in black ink, featuring a large 'R' and 'A' followed by a horizontal line and a small circle.

Raja M. Arkan

A handwritten signature in black ink, appearing to be 'Muhammad Arya P. P.' with a long, sweeping horizontal stroke at the end.

Muhammad Arya P. P.

A handwritten signature in black ink, appearing to be 'Mahesa Satria Prayata' with a cursive, flowing style.

Mahesa Satria Prayata

Teori Singkat

SSS (Shamir Secret Sharing)

Shamir Secret Sharing (SSS) adalah skema kriptografi yang memungkinkan sebuah rahasia dibagi ke dalam beberapa bagian (*shares*) yang kemudian didistribusikan kepada sejumlah pihak sehingga hanya gabungan dari sejumlah minimum shares tertentu yang mampu merekonstruksi rahasia asli.

Konsep utama SSS adalah **threshold** yang dinotasikan sebagai skema (t, n) . Dalam skema ini, sebuah rahasia S dipecah menjadi n buah *share*, namun cukup t *share* ($t \leq n$) yang diperlukan untuk merekonstruksi S . Jika hanya tersedia kurang dari t *share*, maka tidak ada informasi apapun mengenai S yang dapat diperoleh. Sifat ini dikenal sebagai *information-theoretic security* yang berarti penyerang dengan kapasitas komputasi yang luar biasa pun tidak dapat menebak rahasia dari *share* yang tidak mencukupi jumlah **threshold**-nya.

SSS dibangun menggunakan prinsip interpolasi polinomial **Lagrange**, jadi sebuah polinomial berderajat $k - 1$ dapat ditentukan secara unik dari k titik yang berbeda. Penjelasan lebih lanjutnya adalah sebagai berikut:

- Untuk merekonstruksi sebuah garis lurus (polinomial derajat 1), diperlukan tepat 2 titik.

- Untuk merekonstruksi sebuah parabola (polinomial derajat 2), diperlukan tepat 3 titik.
- Secara umum, polinomial berderajat $t - 1$ memerlukan tepat t titik.

Proses pembuatan share dilakukan dengan cara sebagai berikut:

1. Rahasia S disematkan sebagai koefisien konstan $a_0 = S$ dari sebuah polinomial acak berderajat $t - 1$.

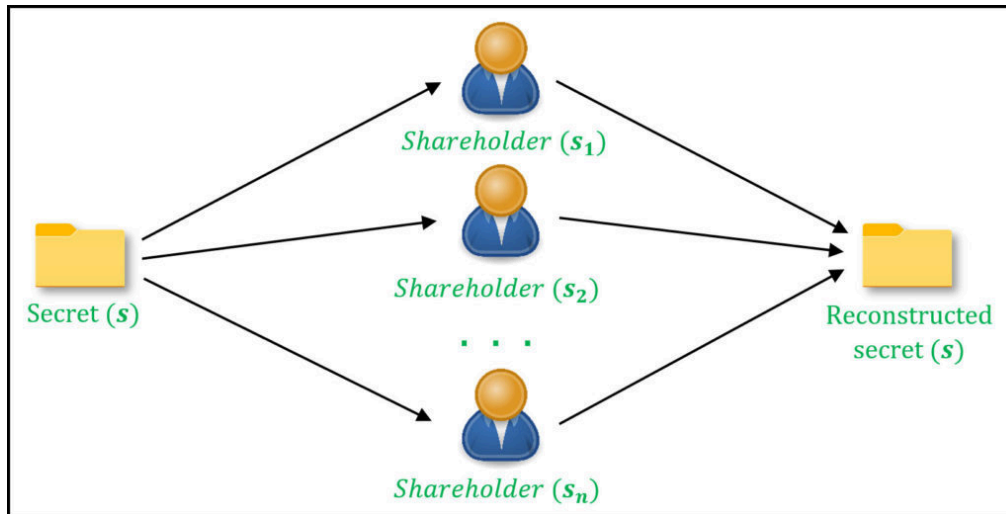
$$f(x) = a_0 + a_1x + a_2x^2 + \dots + a_{t-1}x^{t-1}$$

2. Koefisien $a_1, a_2, \dots, a_{t-1}$ dipilih secara acak.
3. Sebanyak n buah share berhasil dibuat dengan mengevaluasi polinomial pada titik-titik berbeda, *share* ke- i adalah pasangan $(i, f(i))$.

Untuk merekonstruksi rahasia, pemegang minimal t share dapat menggunakan interpolasi Lagrange untuk mendapatkan kembali polinomial $f(x)$, lalu menghitung $f(0) = S$.

Tabel 1.1 Sifat-sifat Shamir Secret Sharing (SSS)

Sifat	Keterangan
<i>Threshold</i> (t, n)	Tepat t dari n jumlah <i>share</i> diperlukan untuk rekonstruksi rahasia
<i>Perfect Secrecy</i>	Kurang dari t <i>share</i> tidak memberikan informasi apapun tentang rahasia
<i>Fault Tolerance</i>	Hilangnya hingga $n - t$ <i>share</i> tidak menghalangi rekonstruksi
Fleksibilitas	<i>Share</i> dapat ditambah atau dihapus tanpa mengubah rahasia asli



Gambar 1.1 Skema Shamir Secret Sharing (SSS)

sumber:

<https://www.geeksforgeeks.org/python/implementing-shamirs-secret-sharing-scheme-in-python/>

AES-GCM

Advanced Encryption Standard (AES) adalah algoritma enkripsi simetris standar yang bekerja pada blok data berukuran 128 bit dan mendukung panjang kunci 128, 192, atau 256 bit. Karena AES merupakan block cipher, penggunaannya selalu melibatkan mode operasi yang menentukan bagaimana blok-blok plaintext diproses secara berurutan.

Mode operasi yang paling umum digunakan dalam sistem modern adalah AES-GCM (*Galois/Counter Mode*). GCM merupakan mode Authenticated

Encryption with Associated Data (AEAD), yaitu mode yang sekaligus memberikan dua jaminan keamanan berupa kerahasiaan data melalui enkripsi dan integritas serta keaslian data melalui *authentication tag*. AES-GCM mengkombinasikan dua komponen utama:

- **Counter Mode**, plaintext dienkripsi dengan men-XOR setiap blok plaintext dengan *keystream* yang dihasilkan dari enkripsi nilai counter yang diinkrementasi secara berurutan. Hasilnya adalah ciphertext.
- **GHASH**, fungsi autentikasi berbasis perkalian di *Glas Field* $GF(2^{128})$ yang memproses ciphertext dan Additional Authenticated Data (AAD) untuk menghasilkan *authentication tag*.

Tabel 1.2 Komponen Input dan Output AES-GCM

Jenis	Komponen	Fungsi
Input	Plaintext	Data asli yang akan dienkripsi
Input	Kunci AES	Kunci simetris untuk operasi enkripsi
Input	IV/Nonce (96 bit)	Nilai unik per operasi enkripsi
Input	AAD	Data yang diautentikasi tetapi tidak dienkripsi
Output	Ciphertext	Hasil enkripsi plaintext
Output	<i>Authentication tag</i>	Nilai verifikasi integritas (umumnya 128 bit)

Pada proses dekripsi, semua komponen (ciphertext, IV, *authentication tag*, dan kunci) harus sesuai. Jika terjadi ketidakcocokan pada *authentication tag*,

proses dekripsi akan langsung dibatalkan dan ciphertext dianggap tidak valid sehingga tidak ada plaintext yang dikembalikan.

KDF (Key Derivation Function)

Key Derivation Function (KDF) adalah fungsi kriptografi yang mengambil suatu bahan kunci dan menghasilkan satu atau lebih kunci kriptografis dengan panjang dan format yang diinginkan. Tujuan utama KDF adalah mengubah suatu sumber entropi seperti *shared secret* dari pertukaran kunci ECDH, sebuah passphrase, atau nilai acak menjadi kunci yang siap digunakan dalam operasi kriptografi lebih lanjut.

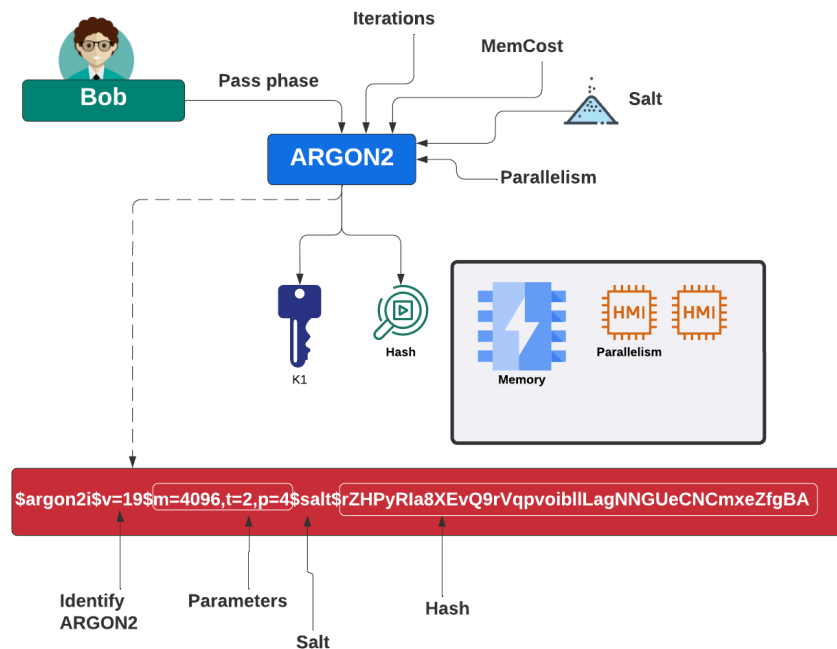
Kebutuhan akan KDF muncul karena kunci yang berasal langsung dari sumber mentah seringkali memiliki distribusi statistik yang tidak seragam atau panjang yang tidak sesuai dengan kebutuhan algoritma tertentu. KDF memastikan kunci turunan memiliki distribusi yang seragam dan panjang yang tepat.

Argon2 adalah pemenang Password Hashing Competition (2015) dan merupakan KDF modern yang digunakan untuk tugas ini. Argon2id bersifat **memory-hard**, yaitu untuk menghitung satu hash penyerang wajib mengalokasikan memori sebanyak 64 MB. Hal ini membuat serangan paralel besar-besaran menjadi sangat mahal karena tiap percobaan password memerlukan memori penuh 64 MB.

Tabel 1.3 Parameter Argon2

Parameter	Nilai	Deskripsi
time_cost	3	Jumlah iterasi
memory_cost	64 MB	Memori yang dialokasikan
parallelism	2	Jumlah lane paralel
hash_len	16 byte	Panjang kunci output (128-bit, sesuai AES-128)
salt	16 byte acak	Menjamin output unik

Salt adalah nilai acak yang dikombinasikan dengan *password* sebelum *hashing*. Dengan *salt*, dua pengguna yang memakai *password* sama akan menghasilkan kunci berbeda dan tabel pra-komputasi menjadi tidak berguna. *Salt* tidak rahasia sehingga disimpan berdampingan dengan data terenkripsi.



Gambar 1.2 Skema Argon2

sumber: <https://medium.com/asecuritysite-when-bob-met-alice/argon2-ffb941a548c6>

CSPRNG (Cryptographically Secure Pseudorandom Number Generator)

Cryptographically Secure Pseudorandom Number Generator (CSPRNG) adalah generator bilangan pseudo-acak yang dirancang khusus untuk memenuhi persyaratan keamanan dalam aplikasi kriptografi. Berbeda dari PRNG biasa yang hanya perlu melewati uji statistik, CSPRNG harus memenuhi syarat keamanan yang jauh lebih ketat. Banyak operasi kriptografi bergantung pada bilangan acak yang berkualitas tinggi, di antaranya:

- pembangkitan kunci simetris dan asimetris,

- pembangkitan IV/nonce untuk enkripsi,
- pembangkitan salt untuk password hashing,
- pembangkitan challenge dalam protokol autentikasi.

Jika PRNG yang digunakan tidak aman secara kriptografis, penyerang yang mampu memprediksi *output*-nya dapat merekonstruksi kunci enkripsi, memalsukan *session token*, atau membatalkan jaminan keamanan protokol secara keseluruhan. Sebuah PRNG disebut *cryptographically secure* apabila memenuhi dua sifat utama berikut:

1. **Next-bit unpredictability**, yaitu saat semua bit output sebelumnya dan tidak ada algoritma yang berjalan dalam waktu polinomial yang dapat memprediksi bit berikutnya dengan probabilitas lebih dari 50%.
2. **Backward secrecy**, jika *internal state generator* berhasil dikompromi oleh penyerang, output yang telah dihasilkan sebelum kompromi tersebut tetap tidak dapat direkonstruksi.

Tabel 1.4 Perbandingan Random Generator

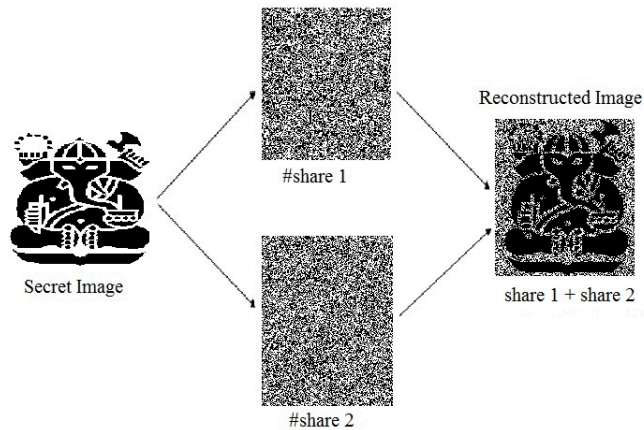
Jenis	Cara Kerja	Kesesuaian untuk Kriptografi
TRNG (<i>True RNG</i>)	Berdasarkan fenomena fisik (noise hardware, dll.)	Sangat baik, namun lambat
PRNG (<i>Pseudo-random</i>)	Deterministik berbasis seed	Tidak cukup, dapat diprediksi
CSPRNG	Deterministik + entropi dari sumber fisik	Sesuai untuk kriptografi

CSPRNG umumnya diimplementasikan dengan mengambil *entropi* dari sumber-sumber fisik sistem operasi (seperti interupsi hardware, timing perangkat, dan noise termal) untuk mengisi *entropy pool* kemudian menggunakannya sebagai seed dalam algoritma.

Kriptografi Visual

Kriptografi visual adalah teknik kriptografi yang memungkinkan penyembunyian informasi visual seperti gambar atau teks ke dalam beberapa buah citra yang disebut *share*. Keunikan kriptografi visual terletak pada proses dekripsinya, yaitu rahasia dapat didapat dengan mata dengan cara menumpuk sejumlah *share* secara fisik atau visual tanpa memerlukan komputasi kriptografis apapun. Untuk tugas kali ini kami akan menggunakan skema dasar $(2, 2)$.

Pada skema $(2, 2)$ sebuah citra rahasia biner hitam putih dipecah menjadi dua *share*. Setiap *share* terlihat seperti *noise* yang tidak mengungkapkan informasi apapun ketika dilihat sendiri-sendiri. Namun ketika kedua *share* ditumpuk, rahasia akan terlihat. Mekanisme *encoding* dengan setiap piksel pada rahasia direpresentasikan oleh sejumlah subpiksel dari masing-masing *share*.



Gambar 1.3 Kriptografi Visual Skema (2,2)

sumber:

<https://catatankriptografi.wordpress.com/2018/02/20/program-kriptografi-visual-untuk-skema-2-2/>

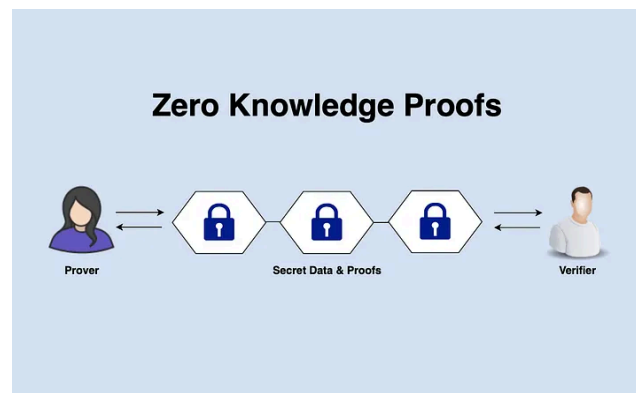
Zero Knowledge Proof

Zero-Knowledge Proof (ZKP) adalah protokol kriptografi yang memungkinkan satu pihak yang disebut *prover* untuk meyakinkan pihak lain yang disebut *verifier* bahwa suatu pernyataan adalah benar tanpa mengungkapkan informasi apapun selain fakta bahwa pernyataan tersebut memang benar. Sebuah protokol dikatakan merupakan zero-knowledge proof jika memenuhi tiga sifat berikut:

Tabel 1.5 Sifat Fundamental Zero Knowledge Proof

Sifat	Keterangan
-------	------------

<i>Completeness</i>	Jika pernyataan yang diklaim benar, maka <i>prover</i> yang jujur akan selalu berhasil meyakinkan <i>verifier</i> yang jujur. Bukti yang valid tidak akan pernah ditolak.
<i>Soundness</i>	Jika pernyataan yang diklaim salah, maka tidak ada <i>prover</i> tidak jujur yang dapat meyakinkan <i>verifier</i> yang jujur kecuali dengan probabilitas yang sangat kecil. Pembuktian palsu tidak dapat dibuat.
<i>Zero-Knowledge</i>	Jika pernyataan yang diklaim benar, maka <i>verifier</i> tidak memperoleh pengetahuan apapun selain fakta bahwa pernyataan tersebut benar. Segala sesuatu yang dapat dipelajari <i>verifier</i> dari interaksinya dengan <i>prover</i> dapat disimulasikan tanpa mengetahui rahasia yang dibuktikan.



Gambar 1.4 Skema Zero Knowledge Proofs

sumber:

<https://medium.com/@darlingtonnnam/beginners-guide-to-understanding-zero-knowledge-proofs-cadc4e2c23a8>

Perancangan dan Implementasi

Arsitektur Umum

Aplikasi ini adalah password manager terdistribusi yang melindungi vault setiap pengguna dengan satu master key 16-byte. Master key tersebut tidak pernah disimpan secara utuh di mana pun dan hanya dapat didapat menggunakan Shamir Secret Sharing skema (2-of-3) menjadi tiga share yang disimpan di tiga lokasi berbeda sebagai berikut:

Tabel 2.1 Tempat Penyimpanan Share

Share	Lokasi Penyimpanan	File yang berhubungan
Local Share	Client (terenkripsi)	<code>src/storage/local_store.py</code>
Server Share	Basis data server	<code>src/server/db.py</code> , <code>src/server/endpoints.py</code>
Recovery Share	Diserahkan ke pengguna (hanya sekali diperlihatkan / kriptografi visual)	<code>src/client/flows/create_vault.py</code>

Karena **threshold** SSS hanya 2, gabungan dua share apa pun sudah cukup untuk merekonstruksi *master key*, sementara satu *share* saja tidak membocorkan informasi apapun tentang *master key*.

Sistem dibagi menjadi dua proses yang berkomunikasi lewat HTTP di 127.0.0.1:5000 saja. Dapat dilihat di file `scripts/run_server.py` dan `scripts/run_client.py` sebagai *entry point*.

Infrastruktur Teknologi

Berikut adalah teknologi yang digunakan untuk membuat aplikasi ini:

Tabel 2.2 Fungsi, Teknologi/Library, dan Alasan Pemilihan Teknologi

Fungsi Utama Sistem	Teknologi / Library	Alasan Pemilihan
Bahasa dan Runtime	Python 3.11+	Python memiliki ekosistem kriptografi yang baik dan mudah dijalankan lintas platform tanpa kompilasi.
Kriptografi	cryptography	Implementasi AES-GCM menggunakan OpenSSL di balik layar sehingga tercatat dan cepat.
	PyCryptodome	Menyediakan implementasi Shamir's Secret Sharing siap pakai di atas $GF(2^8)$.
	argon2-cffi	Menyediakan <code>hash_secret_raw()</code> dengan akses penuh ke parameter time cost, memory cost, dan parallelism dan tahan terhadap serangan GPU karena bersifat <i>memory hard</i> .
Server dan Komunikasi	Flask	Framework yang ringan.
	SQLite	Basis data <i>embedded</i> tanpa konfigurasi tambahan yang cukup untuk menyimpan

		<i>ciphertext</i> dan <i>server share</i> .
	requests	HTTP client paling umum di Python.
Antarmuka Pengguna	Rich	Menghasilkan tampilan terminal berwarna dan terstruktur (tabel, panel, markup) tanpa GUI.
	pyperclip	Memungkinkan fitur copy to clipboard ketika melihat password.
Bonus - Kriptografi Visual	qrcode[pil]	Library QR code paling umum di Python
	Pillow	Library standar untuk manipulasi gambar di Python
	NumPy	Memungkinkan operasi piksel batch (pembuatan pola acak, OR bitwise, reshape)
	OpenCV	Menyediakan <code>cv2.QRCodeDetector</code> untuk membaca QR.
Testing	pytest	Framework testing paling umum di Python.

Struktur Direktori

Kode implementasi `src/` dibagi menjadi beberapa folder agar saat ada hal yang perlu diubah/diperbaiki kedepannya tidak akan susah mencari file-file yang berhubungan. Berikut adalah struktur dari direktori tugas ini:

Tabel 2.3 Struktur Direktori dan Tanggung Jawabnya Masing-Masing

Direktori	Tanggung Jawab
-----------	----------------

<code>src/crypto/</code>	Kriptografi: KDF, AES-GCM, SSS, CSPRNG, format <i>share</i>
<code>src/client/</code>	Client side: flow, UI,
<code>src/server/</code>	Server side bersifat <i>zero knowledge</i> : Flask app, skema DB, <i>endpoint</i>
<code>src/storage/</code>	Data terenkripsi yang disimpan di disk Client
<code>src/bonus/</code>	Kriptografi visual untuk <i>recovery share</i> berbentuk QR
<code>tests/</code>	Uji fungsi krypto (<code>test_aes_gcm.py</code> , <code>test_kdf.py</code> , <code>test_shamir.py</code>)

KDF Argon2id

Implementais ada di file `src/crypto/kdf.py`. *Master password* pengguna tidak langsung dipakai sebagai kunci. Password diturunkan menjadi kdf key 16-byte yang cocok dengan AES-128 menggunakan Argon2id, dan KDF memory-hard yang tahan terhadap serangan GPU/ASIC. KDF key ini bukan *master key vault* dan hanya dipakai untuk mengenkripsi *local share* di *disk client*. *Master key vault* dihasilkan terpisah secara acak. Parameter yang dipilih adalah sebagai berikut:

Tabel 2.4 Parameter Argon2id

Parameter	Nilai	Alasan
<code>time_cost</code>	3	Jumlah iterasi
<code>memory_cost</code>	65536 KB (64 MB)	Membuat serangan paralel mahal

<code>parallelism</code>	2	Disesuaikan dengan desktop pada umumnya
<code>hash_len</code>	16 byte	Sama dengan ukuran kunci AES-128
<code>salt_len</code>	16 byte	<i>Salt</i> acak per pengguna

Fungsi `derive_key()` menghasilkan pasangan `(key, salt)` dengan *salt* acak baru saat pembuatan *vault*, sedangkan `derive_key_from_salt()` menurunkan ulang kunci dengan *salt* yang tersimpan saat *login*. *Salt* disimpan bersama *local share* sehingga kunci dapat direkonstruksi tanpa menyimpan kunci itu sendiri.

AES-128-GCM

Implementasi ada di file `src/crypto/aes_gcm.py`. Seluruh enkripsi memakai AES-128-GCM mode *authenticated encryption* yang memberi kerahasiaan sekaligus *authentication tag*. Rancangan kuncinya adalah sebagai berikut:

- Nonce 12-byte selalu baru dan diambil dari `secrets.token_bytes` pada setiap operasi `encrypt()`. Tidak pernah ada nonce yang dipakai ulang karena setiap re-enkripsi *vault* (saat *create/add/edit/delete*) akan menggunakan *nonce* baru.
- `decrypt()` menampilkan *InvalidTag* bila *ciphertext* atau *nonce* diubah sehingga manipulasi data terdeteksi. Kegagalan dekripsi menandakan failed login (lihat di `src/client/flows/login_normal.py`).
- Panjang kunci divalidasi tepat 16 byte.

Mode ini dipakai untuk mengenkripsi *blob vault* dengan *master key* dan mengenkripsi *local share* dengan kdf key.

Shamir Secret Sharing

Implementasi ada di file `src/crypto/shamir.py`. Implementasi SSS menggunakan *library pycryptodome*. `split_secret()` memvalidasi bahwa rahasia tepat 16 byte (sama dengan master key) dan bahwa $2 \leq \text{threshold} \leq \text{shares}$ lalu memecahnya menjadi daftar pasangan *indeks, share_bytes*. `combine_shares()` merekonstruksi rahasia dan menolak jika diberi kurang dari dua *share*.

CSPRNG Password Generator

Implementasi ada di file `src/crypto/csprng.py`. `generate_password()` membuat *password* acak menggunakan modul *secrets*, bukan *random* biasa. Berikut cara sistem menjamin kualitas *password*:

- Pengguna memilih kategori karakter (huruf besar/kecil, angka, simbol).
- Satu karakter dari tiap kategori terpilih diambil terlebih dahulu dan sisanya diisi dari gabungan seluruh kategori karakter.
- Hasil diacak ulang dengan *Fisher-Yates shuffle* memakai `secrets.randbelow` agar posisi karakter wajib tadi tidak bias.
- Validasi menolak panjang *password* yang lebih kecil dari jumlah kategori terpilih.

Share Format

Implementasi ada di file `src/crypto/share_format.py`. Agar *share* dapat ditampilkan, dikirim lewat JSON, dan dimasukkan kembali oleh pengguna, `encode_share()` membungkus `{index, share}` (*share* di-base64) lalu membungkusnya sekali lagi dengan base64 menjadi satu string ASCII tunggal. `decode_share()` membalik proses ini dengan validasi ketat dan meng-*output* `ValueError` jika string rusak. Hal ini penting karena pengguna mengetik *recovery share* secara manual saat *login backup*.

Alur Pembuatan Vault

Implementasi ini ada di file:

- `src/client/flows/create_vault.py`
- `src/client/main.py`
- `src/client/api_client.py`

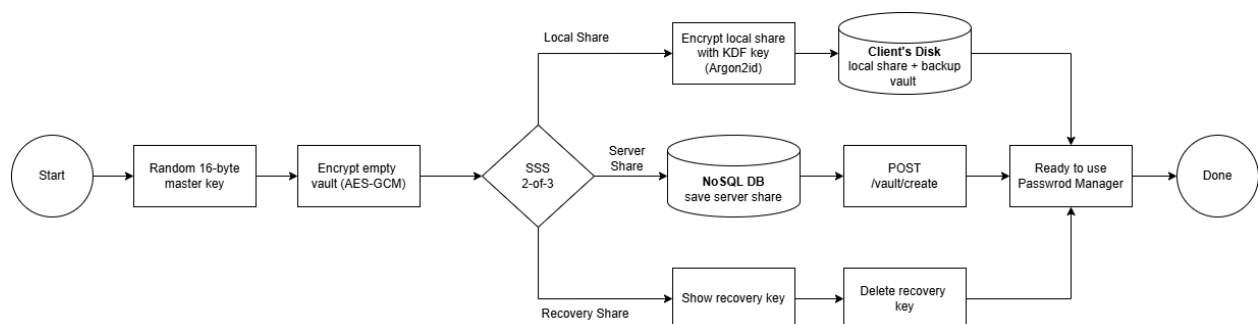
Alur pembuatan vault adalah sebagai berikut:

1. *Generate* master key 16-byte acak (`secrets.token_bytes(16)`) terpisah dari password pengguna.
2. Enkripsi vault kosong dengan *master key* AES-GCM menghasilkan *encrypted_vault* + *vault_nonce*.
3. Pecah *master key* menjadi tiga share dengan SSS, ditetapkan indeks tetap *local* = 0, *server* = 1, *recovery* = 2.
4. Generate KDF key dari *master password* menggunakan Argon2id lalu enkripsi *local share* dengan KDF key tersebut.

5. Kirim data *username*, *server_share*, *encrypted_vault*, dan *vault_nonce* ke server.
6. Simpan data ke disk lokal. *Local share* terenkripsi + *copy backup vault*.
7. Tampilkan *recovery share* ke pengguna yang hanya ditampilkan sekali lalu dibuang. *Recovery share* tidak pernah disimpan ke *disk* maupun dikirim ke server.
8. Pengguna sudah bisa menggunakan Password Manager.

Tabel 2.5 Lokasi Setiap Share Setelah pembuatan Vault

Lokasi	Data yang disimpan
Disk Client	<i>Local share</i> yang terenkripsi KDF key, <i>backup vault</i> , <i>salt</i>
Server	<i>Server share</i> , <i>encrypted vault</i> , <i>nonce</i>
Pengguna	<i>Recovery share</i>



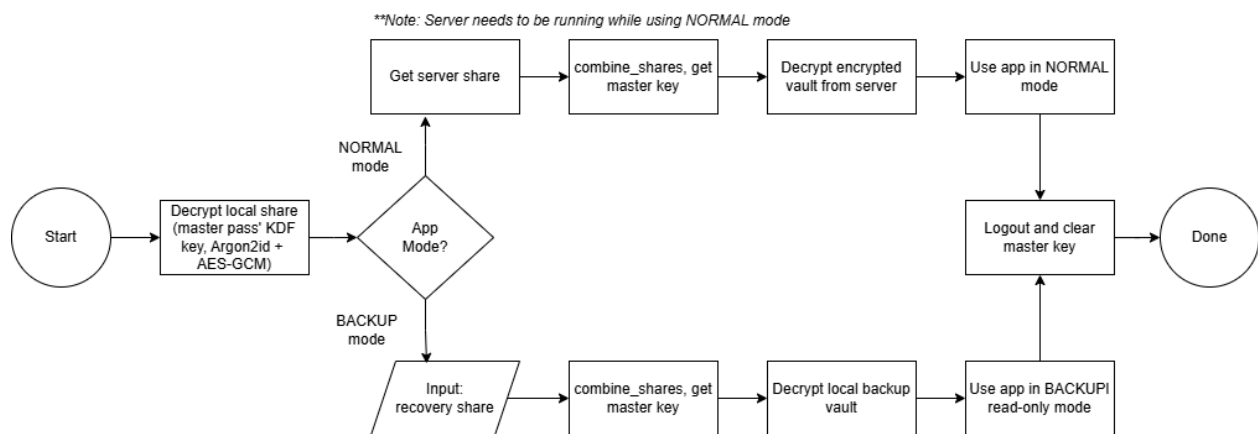
Gambar 2.1 Flowchart Pembuatan Vault

Session State Management

Implementasi ada di file berikut:

- `src/client/state.py`
- `src/client/flows/login_normal.py`
- `src/client/flows/login_backup.py`
- `src/client/flows/vault_ops.py`

SessionState adalah state machine tiga keadaan yaitu LOCKED, NORMAL, dan BACKUP. Master key hanya ada di memori dan tersimpan di `SessionState.master_key`. Saat logout, `clear()` akan menimpa master key dengan 0 untuk setiap byte-nya.



Gambar 2.2 Flowchart Backup dan Normal Login

Mode backup bersifat *read-only*. Setiap fungsi mutasi di `vault_ops.py` (`add_password`, `edit_password`, `delete_password`) memeriksa `session.is_read_only()` terlebih dahulu dan tidak jika *True*.

`vault_ops.py` juga menerapkan pola *persist-atau-rollback*, yaitu perubahan vault hanya dianggap berhasil jika `_persist()` (enkripsi ulang + kirim ke

server + simpan backup) sukses. Jika gagal, perubahan di memori dibatalkan.

Penyimpanan Lokal

Implementasi ada di file `src/storage/local_store.py`. Disk client menyimpan dua file di `data/client/` sebagai berikut:

Tabel 2.6 File yang disimpan di Lokal

File	Isi	Fungsi
<code>local_share.enc</code>	<i>local share</i> terenkripsi, <i>nonce</i> , <i>salt</i> KDF, parameter KDF	Bahan login untuk <i>login normal</i> dan <i>login backup</i>
<code>backup_vault.enc</code>	<i>copy</i> dari <i>encrypted vault</i> + <i>nonce</i>	Membuka <i>vault offline</i> dan <i>read-only</i>

Keduanya berformat JSON dengan nilai biner ter-base64. *Master key*, *kdf key*, dan *recovery share* tidak pernah menyentuh disk.

Server Zero Knowledge

Implementasi ada di file berikut:

- `src/server/app.py`
- `src/server/db.py`
- `src/server/endpoints.py`

Server dirancang *zero knowledge* sehingga tidak mengetahui isi *vault* dan hanya menyimpan *encrypted vault*. Skema SQLite hanya ada 2 tabel dengan data sebagai berikut:

1. `users(id, username, created_at)`
2. `vaults(id, user_id, server_share, encrypted_vault, vault_nonce, updated_at)`

Server tidak menyimpan password, key, dan plaintext. Berikut adalah endpoint yang disediakan:

Tabel 2.7 Dokumentasi endpoint

endpoint	Fungsi	Validasi
<code>GET /health</code>	Cek keadaan server	-
<code>POST /vault/create</code>	Daftar pengguna + save vault	field wajib, base64 valid, nonce 12-byte, username unik (409 bila ada)
<code>GET /vault/<username></code>	Ambil server share + encrypted vault	error 404 jika tidak ada
<code>PUT /vault/<username></code>	<i>Update</i> encrypted vault	base64 valid, nonce 12-byte

Server hanya memvalidasi bentuk data (base64, panjang nonce), bukan isinya.

Antarmuka Pnegguna (CLI)

Implementasi ada di file berikut:

- `src/client/main.py`
- `src/client/ui/menu.py`
- `src/client/ui/prompts.py`
- `src/client/vault.py`

UI berbasis terminal menggunakan *library* Rich. `main.py` berperan sebagai *controller* yang menampilkan *menu*, memanggil *flow* yang sesuai, lalu masuk ke `_vault_loop` untuk melakukan operasi CRUD. Menu *vault* secara otomatis menyembunyikan aksi CRUD ketika dalam sesi *read-only* (parameter `read_only` pada `show_vault_menu`).

Bonus – Kriptografi Visual

Implementasi ada di file `src/bonus/visual_crypto.py`. *Recovery share* dapat diamankan dengan **Visual Cryptography skema (2,2)**. *Recovery share* diubah menjadi QR code, lalu QR tersebut dipecah menjadi dua gambar PNG berupa *noise*. QR akan terbaca hanya ketika kedua gambar ditumpuk baik secara fisik atau secara digital dengan operasi OR. Berikut adalah implementasi visual kriptografi:

Tabel 2.8 Implementasi Kriptografi Visual

Tahap	Code Function	Deskripsi
-------	---------------	-----------

Encode QR	<code>recovery_share_to_qr()</code>	Recovery share dirender jadi QR.
Pengubahan menjadi biner	<code>_to_bin()</code> / <code>_to_pil()</code>	Konversi gambar ke matriks biner dan sebaliknya (1 = piksel gelap)
Pemecahan	<code>split_qr_visual()</code>	Setiap piksel QR dipetakan ke sepasang sub-piksel dengan pola acak
Save .png	<code>save_shares()</code>	Menyimpan file share1.png dan share2.png di <code>\data\client</code>

Inti logika dari `split_qr_visual()` adalah pixel expansion, jadi tiap piksel QR menjadi dua subpiksel. Berikut adalah penjelasannya lebih lanjut:

- Sebuah pola acak pick (0/1) di-generate per piksel. *Share* pertama mendapat pasangan $[pick, 1-pick]$.
- Untuk piksel terang QR, *share* kedua memakai pola identik dengan *share* pertama sehingga saat di-OR hasilnya sesuai.
- Untuk piksel gelap QR, *share* kedua memakai pola komplemen ($1 - pattern_a$) sehingga saat di-OR kedua subpiksel terlihat gelap.

Dengan demikian, masing-masing *share* mempunyai distribusi subpiksel acak sehingga tidak membocorkan struktur QR.

Pengujian Program dan Analisis

Uji Pembuatan Vault

Untuk pengujian pertama akan dilakukan pembuatan vault baru dengan *master password* valid.

```
Distributed Password Manager (Shamir SSS)

== Password Manager ==
1) Create vault
2) Login (Normal)
3) Login (Backup)
4) Quit
Choose [1/2/3/4] (2): 1
Username: login@mail.com
Master password:
Confirm master password:
Creating vault (this takes a moment for key derivation)...
Encrypting local share...
✓ Local share encrypted successfully.
✓ Vault created successfully!

----- Recovery Share -----
eyJpbmRleCI6IDMsICJzaGFyZSI6ICI3Sk41QUU1Y1ROVXNnZHNkdmlncGt3PT0ifQ==
SAVE THIS RECOVERY SHARE NOW.
It will not be shown again. Without it, you cannot recover your vault if the server is unavailable.

Apakah mau mengenerate QR? [y/n] (n): y

=====
[ALUR BONUS] Pemrosesan Citra QR Code untuk user: login@mail.com
=====
✓ QR Code utama berhasil dibuat.
Apakah Anda ingin MENYIMPAN gambar QR utuh/jadi ini ke folder vc_user? [y/n] (n): y
✓ Berkas QR utama disimpan di: data\client\vc_login@mail.com\recovery_qr_original.png

[PROSES] Memecah citra QR utama menjadi visual shares terdistribusi...
[OTOMATIS] Komponen Visual Cryptography selalu dibuat & disimpan:
-> data\client\vc_login@mail.com\share1.png
-> data\client\vc_login@mail.com\share2.png
✓ Visual cryptography split shares berhasil diamankan ke folder lokal.
=====

Recommended visual share handling:
1. Print both PNGs on paper or transparency sheets.
2. Store the two printouts in different physical locations
   (e.g., one at home, one with a trusted person).
3. Delete the digital PNG files after printing, because keeping
   both files together defeats the purpose of splitting.
4. To recover, physically overlay the two prints, scan the
   resulting QR with any phone, then paste the decoded
   string into Login (Backup) as your recovery share.

You can now login using Normal Mode.

== Password Manager ==
1) Create vault
2) Login (Normal)
3) Login (Backup)
4) Quit
Choose [1/2/3/4] (2):
```

Gambar 3.1 Pembuatan vault baru

Pengujian selanjutnya akan menunjukkan master key dibagi menjadi tiga share menggunakan Shamir Secret Sharing dengan skema (2, 3), yaitu local share, server share, dan recovery share.

```
✓ Vault created successfully!

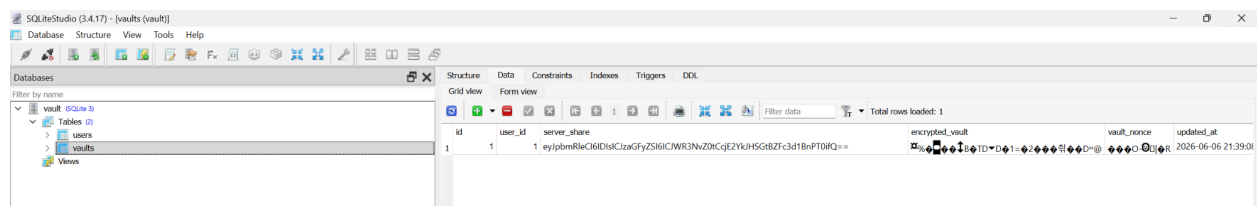
Recovery Share

eyJpbmRleCI6IDMsICJzaGFyZSI6ICI3Sk41QUU1Y1ROVXNnZHNkdm1ncGt3PT0ifQ==

SAVE THIS RECOVERY SHARE NOW.
It will not be shown again. Without it, you cannot recover your vault if the server is unavailable.

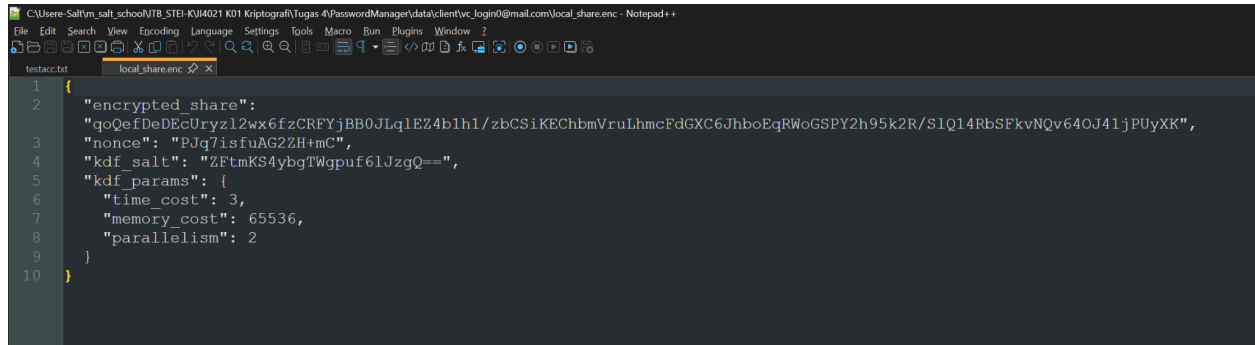
Apakah mau generate QR? [y/n] (n): y
```

Gambar 3.2 Recovery Share



id	user_id	server_share	encrypted_vault	vault_nonce	updated_at
1	1	eyJpbmRleCI6IDMsICJzaGFyZSI6ICI3Sk41QUU1Y1ROVXNnZHNkdm1ncGt3PT0ifQ==			2026-06-06 21:39:01

Gambar 3.3 Server Share



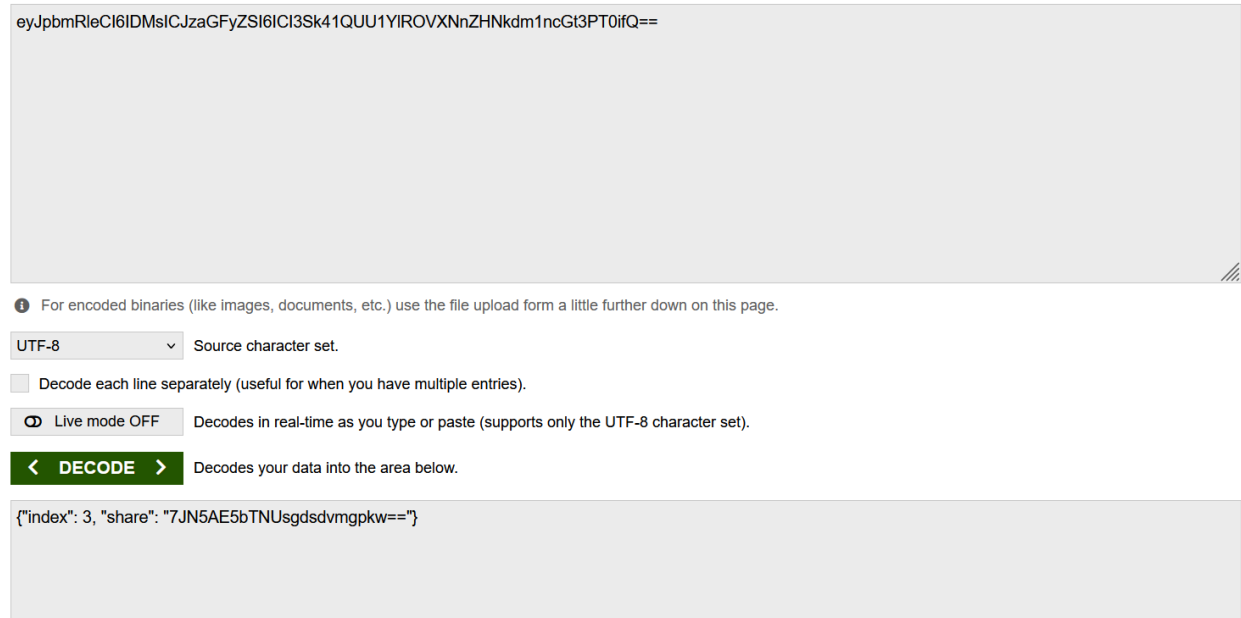
```
1 {
2   "encrypted share":
3   "goQefDeDECUryz12wx6fzCRFYjBB0JLq1EZ4b1h1/zbCSiKEChbmVruLhmcFdGXC6JhboEqRwoGSPY2h95k2R/S1Q14RbSFkvNQv640J41jPUyXK",
4   "nonce": "Pjq7isfuAG2ZH+mC",
5   "kdf_salt": "ZFtmKS4ybgTWgpuf6lJzgQ==",
6   "kdf_params": {
7     "time_cost": 3,
8     "memory_cost": 65536,
9     "parallelism": 2
10 }
```

Gambar 3.4 Local Share

Pengujian selanjutnya akan menunjukkan recovery share ditampilkan dalam format teks yang memuat koordinat share dan nilai share.

Decode from Base64 format

Simply enter your data then push the decode button.



eyJpbmRleCI6IDMsICJzaGFyZSI6ICI3Sk41QUU1YiROVXNnZHNkdm1ncGt3PT0ifQ==

For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8 Source character set.

☐ Decode each line separately (useful for when you have multiple entries).

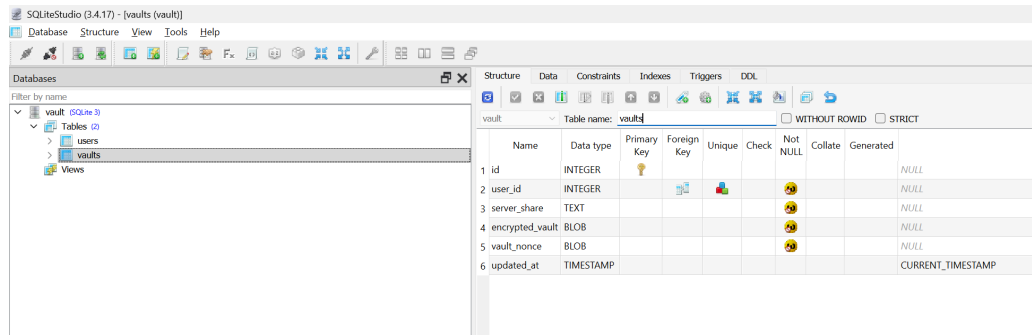
☒ Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

< DECODE > Decodes your data into the area below.

```
{\"index\": 3, \"share\": \"7JN5AE5bTNUsgdsdvmgpkw==\"}
```

Gambar 3.5 Recovery Share Memuat Koordinat Share dan Nilai Share

Server hanya menyimpan server share, vault terenkripsi, nonce vault, dan metadata yang diperlukan.



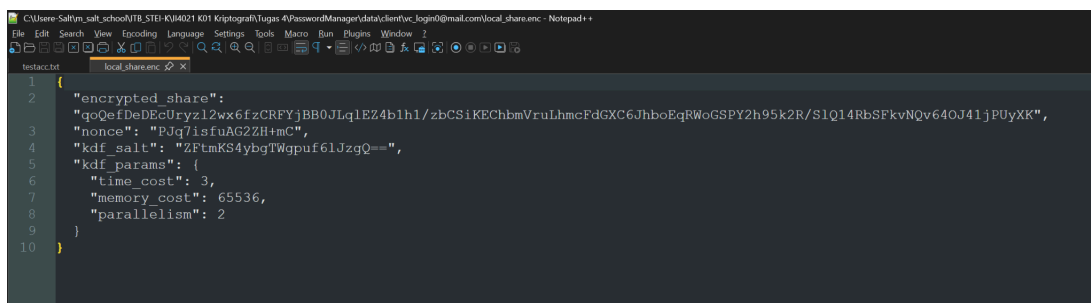
The screenshot shows the SQLiteStudio interface with the 'vault' table selected. The table structure is as follows:

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated
1	id	INTEGER	Yes						NULL
2	user_id	INTEGER							NULL
3	server_share	TEXT							NULL
4	encrypted_vault	BLOB							NULL
5	vault_nonce	BLOB							NULL
6	updated_at	TIMESTAMP							CURRENT_TIMESTAMP

Gambar 3.6 Data yang Disimpan Server

Uji penyimpanan dan perlindungan *local share*

Local share tidak disimpan dalam bentuk asli, melainkan disimpan dalam bentuk base 64.



The screenshot shows a Notepad++ window with a file named 'local_share.enc'. The file contains a JSON object representing a local share, where the 'encrypted_share' value is a long base64-encoded string.

```
1 {
2   "encrypted_share":
3     "qoQefDeDeUryz12wx6fzCRFYjBB0JLq1EZ4b1h1/zbCSiKEChbmVruLhmcFdGXC6JhboEgRwoGSPY2h95k2R/S1Q14RbSFkvNqv640J41jPUyXK",
4   "nonce": "Pjg7isfuAG2ZH+mC",
5   "kdf_salt": "zFtmKS4ybgTWgpuf61JzgQ==",
6   "kdf_params": {
7     "time_cost": 3,
8     "memory_cost": 65536,
9     "parallelism": 2
10  }
```

Gambar 3.7 Local Share disimpan dalam bentuk base 64

Perlihatkan bahwa local share berhasil dienkripsi menggunakan kunci turunan dari master password.

```
Creating vault (this takes a moment for key derivation)...
Encrypting local share...
✓ Local share encrypted successfully.
✓ Vault created successfully!
Recovery Share
```

Gambar 3.8 Local Share Berhasil Dienkripsi

Pengujian selanjutnya akan memasukkan master password yang benar, local share berhasil didekripsi.

```
== Password Manager ==
1) Create vault
2) Login (Normal)
3) Login (Backup)
4) Quit
Choose [1/2/3/4] (2): 2
Username: login0@mail.com
Master password:
Logging in...
Decrypting local share...
✓ Local share decrypted and decoded successfully.
Reconstructing vault master key from shares...
✓ Successfully authenticated as login0@mail.com (Normal Mode).
✓ Logged in as 'login0@mail.com' (normal mode).

== Vault Menu == (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1):
```

Gambar 3.9 Local Share Berhasil Didekripsi

Selanjutnya akan memasukan master password yang salah, local share gagal didekripsi atau akses ditolak.

```
== Password Manager ==
1) Create vault
2) Login (Normal)
3) Login (Backup)
4) Quit
Choose [1/2/3/4] (2): 2
Username: login0@mail.com
Master password:
Logging in...
Decrypting local share...
Error: Login Failed: Password salah atau integritas data rusak. ()
Error: Login failed. Check username and master password.

== Password Manager ==
1) Create vault
2) Login (Normal)
3) Login (Backup)
4) Quit
Choose [1/2/3/4] (2):
```

Gambar 3.10 Local Share Gagal Didekripsi

Uji akses normal

Vault akan dibuka dengan master password yang benar dan server dapat diakses. Sistem akan menggunakan kombinasi local share dan server share.

```

== Password Manager ==
1) Create vault
2) Login (Normal)
3) Login (Backup)
4) Quit
Choose [1/2/3/4] (2): 2
Username: login0@mail.com
Master password:
Logging in...
Decrypting local share...
✓ Local share decrypted and decoded successfully.
Reconstructing vault master key from shares...
✓ Successfully authenticated as login0@mail.com (Normal Mode).
✓ Logged in as 'login0@mail.com' (normal mode).

== Vault Menu == (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1):

```

Gambar 3.11 Local Share Digunakan

```

[SERVER STATUS] Authentication fetch requested. Retrieving remote cryptographic shares for user 'login0@mail.com'...
[SERVER SUCCESS] Dispatched server share and vault payload successfully for user 'login0@mail.com'. Client can now process decryption.
127.0.0.1 - - [07/Jun/2026 05:51:26] "GET /vault/login0@mail.com HTTP/1.1" 200 -
127.0.0.1 - - [07/Jun/2026 05:58:32] "GET /health HTTP/1.1" 200 -

```

Gambar 3.12 Server Share Dipakai Dengan Local Share

master key berhasil direkonstruksi dari dua share valid.

Vault berhasil didekripsi dan isi vault dapat ditampilkan.

```

Username: login@mail.com
Master password:
Logging in...
Decrypting local share...
✓ Local share decrypted and decoded successfully.
Reconstructing vault master key from shares...
✓ Successfully authenticated as login@mail.com (Normal Mode).
✓ Logged in as 'login@mail.com' (normal mode).

== Vault Menu == (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1): 1
(vault is empty)

== Vault Menu == (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1): _

```

Gambar 3.13 Isi Vault Kosong Dapat Dilihat

Master password yang salah, dekripsi vault gagal dan data password tidak ditampilkan.

```

== Password Manager ==
1) Create vault
2) Login (Normal)
3) Login (Backup)
4) Quit
Choose [1/2/3/4] (2): 2
Username: login@mail.com
Master password:
Logging in...
Decrypting local share...
Error: Login Failed: Password salah atau integritas data rusak. ()
Error: Login failed. Check username and master password.

== Password Manager ==
1) Create vault
2) Login (Normal)
3) Login (Backup)
4) Quit
Choose [1/2/3/4] (2): _

```

Gambar 3.14 Vault Tidak Dapat Diakses

Uji Penambahan Data Password

Pengujian selanjutnya akan menambahkan data password secara manual.

Data tersebut akan tersimpan ke dalam vault.

```
== Vault Menu == (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1): 3
Service name: github
Account username: login0
Password source [manual/generate] (generate): manual
Password:
Notes (optional): LGONA
✓ Added entry for 'github'.

== Vault Menu == (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1): 1
Vault Entries


| # | Service | Username | Password | Notes |
|---|---------|----------|----------|-------|
| 1 | github  | login0   | *****    | LGONA |



== Vault Menu == (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1): 2

1) github - login0
0) Cancel
Select entry [0/1] (0): 1

Service: github
Username: login0
Password: LOGIN101
Notes: LGONA

Apakah Anda ingin menyalin password ini ke clipboard? [y/n] (n): n
Penyalinan ke clipboard dibatalkan.
```

Gambar 3.15 Penambahan Data Password

Selanjutnya akan menambahkan data password dengan pembangkitan otomatis menggunakan CSPRNG.


```

===== Vault Menu ===== (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1): 3
Service name: github
Account username: login0
Password source [manual/generate] (generate): generate
Length (20): 14
Generated password: G{2j7lGb,}Uw5g
Notes (optional): fd safe
✓ Added entry for 'github'.

===== Vault Menu ===== (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1): 1
Vault Entries

# | Service | Username | Password | Notes |
--|---|---|---|---|
1 | github | login0 | ..... | LGONA |
2 | github | login0 | ..... | fd safe |

===== Vault Menu ===== (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1): 2

1) github - login0
2) github - login0
0) Cancel
Select entry [0/1/2] (0): 2

Service: github
Username: login0
Password: G{2j7lGb,}Uw5g
Notes: fd safe

```

Gambar 3.16 Penambahan Data Password Dengan CSPRNG

Generasi password tersebut sesuai dengan panjang yang diminta.

Penambahan data, vault dienkripsi ulang menggunakan nonce baru dan disimpan kembali ke server.

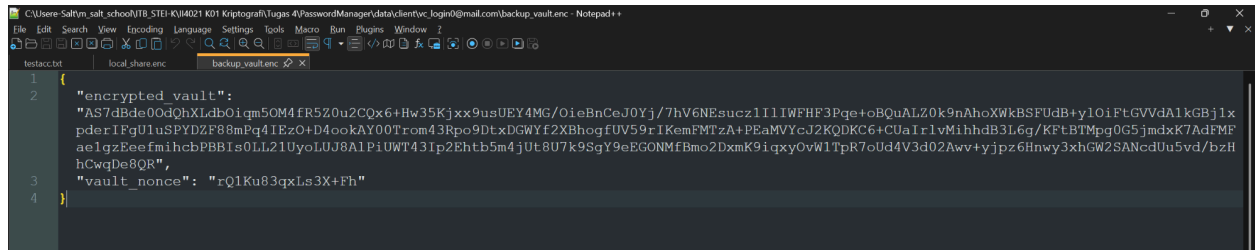
```

[SERVER STATUS] Initiating data modify request for user 'login0@mail.com'...
[SERVER STATUS] Modifying encrypted data block and initialization parameters for user 'login0@mail.com'...
[SERVER SUCCESS] Server vault database updated. New cryptogram and updated nonces saved successfully for user 'login0@mail.com'.
[SERVER INFO] Client must now proceed to sync and save local backup configurations.
127.0.0.1 - - [07/Jun/2026 06:28:37] "PUT /vault/login0@mail.com HTTP/1.1" 200 -

```

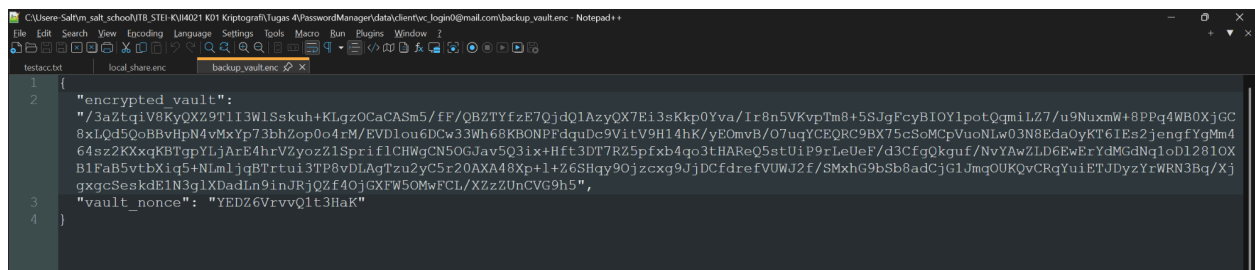
Gambar 3.17 Enkripsi Ulang Vault Dengan Nonce Baru

Pada backup vault lokal juga diperbarui setelah perubahan pada mode normal.



```
1 {
2   "encrypted_vault":
3     "AS7dBde00dQhXLdb0iqm50M4fR5Z0u2CQx6+Hw35Kjxx9usUEY4MG/OieBnCeJ0Yj/7hV6NEsucz1I1IWfHF3Pge+oBQuALZ0k9nAhoXWkBSFudB+y1oiFtGVVdAlkGBj1x
4     pderIFgUluSPYDZF88mPq4IEz0+D4ookAY00Trom43Rpo9DtxDGWYf2XBhogfUV59rIKemFMTzA+PEaMVYcJ2KQDKC6+CUaIr1vMihhdB3L6g/KftBTMpg0G5jmdxK7AdFMF
      ae1gzEeefmihcbPBBIs0LL21UyoLUJ8A1PiUWT43Ip2Ehtb5m4jUt8U7k9SgY9eEGONMEBmo2DxmK9iqxy0vW1TpR7oUd4V3d02Awv+yjz6Hnwy3xhGW2SANcdUu5vd/bzH
      hCwqDe8QR",
5     "vault_nonce": "rQ1Ku83qxLs3X+Fh"
6   }
7 }
```

Gambar 3.18 Backup Vault Sebelumnya



```
1 {
2   "encrypted_vault":
3     "/3aZtqiV8KyQXZ9T1I3W1Sskuh+KLgzOCaCAsm5/FF/QBZTYfzE7QjdQ1AzyQX7Ei3sKkp0Yva/Ir8n5VKvpTm8+5SJgFcyBIOY1potQqmiLZ7/u9NuxmW+8PPq4WB0XjGC
4     8xLQd5QoBBvHpN4vMxYp73bhZop0o4rM/EVDlou6DCw33Wh68KBONPFdquDc9VItV9H14hK/yEOmvB/O7uqYCEQRC9BX75cSoMCpVuoNlw03N8EdaOyKT6IEs2jengfYgMm4
      64sz2KXxqKBTgpYlJArE4hrVZyozZ1Spri f1CHWgCN5OGJav5Q3ix+Hft3DT7RZ5pfb4qo3tHAREQ5stUiP9rLeUeF/d3CFgQkqkf/NvYAwZLD6EwErYdMGdNq1oD1281OX
      B1FaB5vtbXiq5+Nlm1jgBTtui3TP8vDLAgTzu2yC5r20AXA48Xp+1+Z6SHqy90jzcxg9JjDCfdrefVUWUJ2f/SMxhG9bSb8adCjG1JmqOUKQvCRqYuiETJDyzYrWRN3Bq/Xj
      gxgcSeskdE1N3glXdadLn9inJRjQZf40jGXFw5OMwFCL/XZzZUnCVG9h5",
5     "vault_nonce": "YEDZ6VrvvQlt3HaK"
6   }
7 }
```

Gambar 3.19 Backup Vault Setelah

Uji Pengubahan dan Penghapusan Data Password

Salah satu data password yang tersimpan akan diubah. Perubahan tersebut akan tersimpan setelah vault dienkripsi ulang.

```

== Vault Menu == (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1): 4

1) github - login0
2) guthub - login0
3) gethub - login0
0) Cancel
Select entry [0/1/2/3] (0): 1
Editing 'github'. Press Enter to keep current value.
Service : github
Username : login00
Change password? [y/n] (n): y
Password source [manual/generate] (generate): manual
Password:
Notes [LGONNA]: LGONNNA
✓ Entry updated.

== Vault Menu == (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1): 2

1) github - login00
2) guthub - login0
3) gethub - login0
0) Cancel
Select entry [0/1/2/3] (0): 1

Service: github
Username: login00
Password: lgojikoko
Notes: LGONNNA

Apakah Anda ingin menyalin password ini ke clipboard? [y/n] (n): n
Penyalinan ke clipboard dibatalkan.

== Vault Menu == (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1): _

```

Gambar 3.20 Password diubah setelah diedit

```

127.0.0.1 - - [07/Jun/2026 14:35:41] "PUT /vault/login0@mail.com HTTP/1.1" 200 -
[SERVER STATUS] Initiating data modify request for user 'login0@mail.com'...
[SERVER STATUS] Modifying encrypted data block and initialization parameters for user 'login0@mail.com'...
[SERVER SUCCESS] Server vault database updated. New cryptogram and updated nonces saved successfully for user 'login0@mail.com'.
[SERVER INFO] Client must now proceed to sync and save local backup configurations.
127.0.0.1 - - [07/Jun/2026 14:51:53] "PUT /vault/login0@mail.com HTTP/1.1" 200 -

```

Gambar 3.21 Vault Berhasil Dienkripsi Ulang

Salah satu data password yang tersimpan akan dihapus. Data tersebut tidak muncul lagi setelah vault dibuka kembali.

```
Choose [1/2/3/4/5/6] (1): 1
Vault Entries
# Service Username Password Notes
1 github login00 ***** LGOMMNA
2 guthub login0 ***** fd safe
3 gethub login0 ***** jfjffj

Vault Menu == (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1): 5

1) github - login00
2) guthub - login0
3) gethub - login0
0) Cancel
Select entry [0/1/2/3] (0): 3
Permanently delete entry for 'gethub'? [y/n] (n): y
Deleted 'gethub'.

Vault Menu == (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1): 1
Vault Entries
# Service Username Password Notes
1 github login00 ***** LGOMMNA
2 guthub login0 ***** fd safe

Vault Menu == (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1):
```

Gambar 3.22 Data Password Berhasil Dihapus

```
[SERVER STATUS] Initiating data modify request for user 'login0@mail.com'...
[SERVER STATUS] Modifying encrypted data block and initialization parameters for user 'login0@mail.com'...
[SERVER SUCCESS] Server vault database updated. New cryptogram and updated nonces saved successfully for user 'login0@mail.com'.
[SERVER INFO] Client must now proceed to sync and save local backup configurations.
127.0.0.1 - - [07/Jun/2026 15:05:51] "PUT /vault/login0@mail.com HTTP/1.1" 200 -
```

Gambar 3.23 Vault Berhasil Dienkripsi Ulang

Pembaruan dan perubahan data password hanya dapat dilakukan pada mode normal. Pada mode backup, tidak ada opsi untuk melakukan pembaruan dan perubahan data password.

```
===== Vault Menu ===== (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1):
Vault Entries

# | Service | Username | Password | Notes
--|---|---|---|---
1 | github | login00 | ..... | LGONNNA
2 | guthub | login0  | ..... | fd safe

===== Vault Menu ===== (normal mode)
1) List entries
2) Reveal entry password
3) Add entry
4) Edit entry
5) Delete entry
6) Logout
Choose [1/2/3/4/5/6] (1): _
```

Gambar 3.24 Menu pada Mode Normal

```
===== Password Manager =====
1) Create vault
2) Login (Normal)
3) Login (Backup)
4) Quit
Choose [1/2/3/4] (2): 3
Username: login0@mail.com
Paste password:
Paste the recovery share string (single line):
Recovery share: eyJpbmRleCI6IDMsICJzaGFyZSI6ICI3Sk41QUU1VlROVXNnZHNkdmlncGt3PT0ifQ==
Restoring vault from backup...
Decrypting local share...
✓ Local share decrypted successfully.
Combining shares via Shamir Secret Sharing...
✓ Successfully authenticated as login0@mail.com (Read-Only Backup Mode).
✓ Logged in as 'login0@mail.com' (backup / read-only).

===== Vault Menu ===== (READ-ONLY backup mode)
1) List entries
2) Reveal entry password
6) Logout
Choose [1/2/6] (1):
Vault Entries

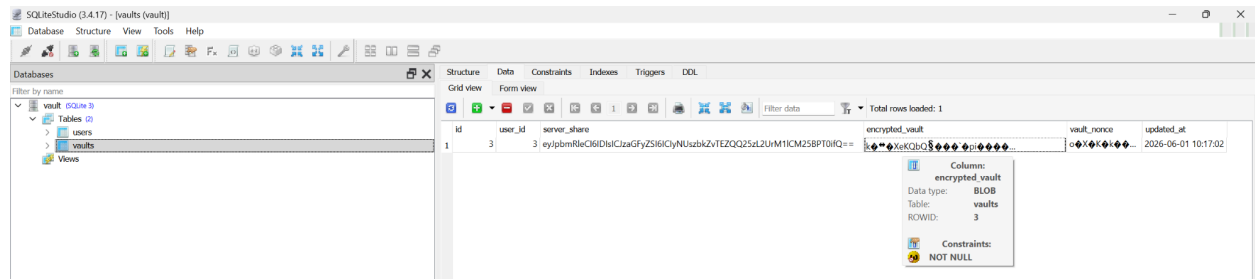
# | Service | Username | Password | Notes
--|---|---|---|---
1 | github | login00 | ..... | LGONNNA
2 | guthub | login0  | ..... | fd safe

===== Vault Menu ===== (READ-ONLY backup mode)
1) List entries
2) Reveal entry password
6) Logout
Choose [1/2/6] (1): 4
Please select one of the available options
Choose [1/2/6] (1): 3
Please select one of the available options
Choose [1/2/6] (1):
```

Gambar 3.25 Menu pada Mode Backup

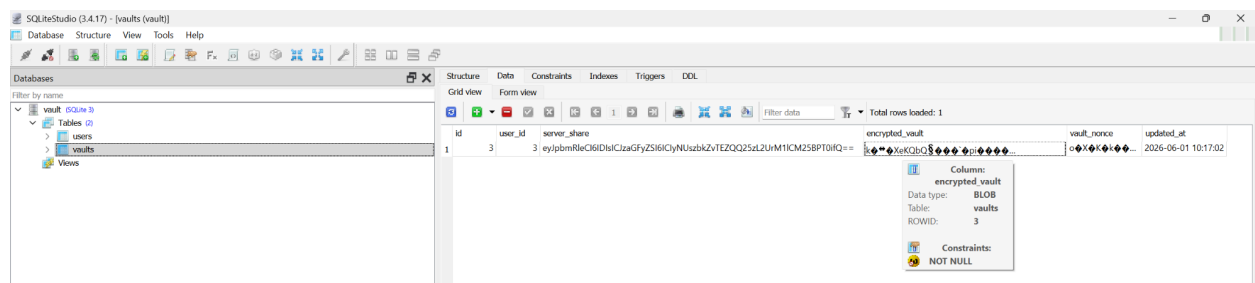
Uji Penyimpanan Vault di Server

Vault di database SQLite tersimpan sebagai dalam bentuk format BLOB.



Gambar 3.26 Data Vault disimpan dalam bentuk BLOB

Server menyimpan data Vault dalam bentuk terenkripsi sehingga nama layanan, username, password, atau catatan tidak dapat dibaca secara plaintext.



Gambar 3.27 Data Vault disimpan terenkripsi

Server tidak menerima master key, local share, recovery share, maupun isi vault dalam bentuk plaintext. Untuk membuktikan ini akan digunakan test script yang membuktikan server tidak menerima plaintext.

```
def prove_zero_knowledge(username: str):
    print("=== [UJI KASUS] Verifikasi Skema Zero-Knowledge ===")

    # 1. Ambil data dari API Server untuk melihat apa yang ditransmisikan di jaringan
    print("[INFO] Mengambil payload vault dari endpoint server...")
    server_data = api_client.get_vault(username)

    if not server_data:
        print(f"[GAGAL] Data untuk user '{username}' tidak ditemukan di server.")
        return

    print("\n--- Payload Transmisi Jaringan (Intercepted) ---")
    print(f"Server Share (Encoded String) : {server_data['server_share']}")
    print(f"Encrypted Vault (Base64)      : {(base64.b64encode(server_data['encrypted_vault']).decode())}")
    print(f"Vault Nonce (Base64 IV)          : {(base64.b64encode(server_data['vault_nonce']).decode())}")

    # 2. Periksa langsung ke dalam penyimpanan piringan lokal SQLite Server
    print("\n[INFO] Memeriksa isi database lokal SQLite Server secara langsung...")
    if not Path(DB_PATH).exists():
        print(f"[GAGAL] File database tidak ditemukan di jalur: {DB_PATH}")
        return

    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    try:
        row = conn.execute(
            "SELECT v.* FROM vaults v JOIN users u ON u.id = v.user_id WHERE u.username = ?",
            (username,)
        ).fetchone()

        print("\n--- Record Data Asli di Dalam File Database (Disk) ---")
        print(f"Kolom 'server_share' : {row['server_share']}")
        print(f"Kolom 'encrypted_vault': {row['encrypted_vault'][:20]}... (Biner Terenkripsi/BLOB)")
        print(f"Kolom 'vault_nonce'   : {row['vault_nonce'].hex()} (Hex Nonce)")

        # Skenario pembuktian: Coba decode BLOB terenkripsi ke teks biasa
        try:
            row['encrypted_vault'].decode('utf-8')
            print("[PERINGATAN] Data tersimpan dalam bentuk teks biasa (Plaintext)!")
        except UnicodeDecodeError:
            print("[BERHASIL] Konfirmasi: Isi data vault berupa biner acak mentah (BLOB).")
            print("[BERHASIL] Konfirmasi: Server TIDAK menyimpan master_key, local_share, ataupun plaintext isi vault.")

    finally:
        conn.close()
```

Gambar 3.28 Test Script yang Dipakai

```
C:\Usere-Salt\m_salt_school\ITB_STEI-K\II4021 K01 Kriptografi\Tugas 4\PasswordManager>PYTHON TESTS\test_zero_knowledge.py
=== [UJI KASUS] Verifikasi Skema Zero-Knowledge ===
[INFO] Mengambil payload vault dari endpoint server...

--- Payload Transmisi Jaringan (Intercepted) ---
Server Share (Encoded String) : eyJpbmRleCI6IDIsICJzaGFyZSI6ICJWR3NvZ0tCcjE2YkJSbGtBZFc3d1BnPT0ifQ==
Encrypted Vault (Base64)      : g+0ai6GULWwxS4dzQX9K7z1lAZcip0vO+riQFlsZpaUhtFvN2DIzbK7TLuhdvA6FIjyc1gC24VlYgujFZX5
2kQ6a4H7/sxU5suue74Nk3yCmm0ti1MsfmLEKDVytVcztpQAvHBR07pRFPWyD90zZ95Hp4yVWTEamcx1Ltgdj+M0ehYyi7ysH1wzakt6bN+Mpp+VRZD
RLjx1RZTPAZMK0zX51MNU+PSGw8sSKWYktEQG+cavu0525AFe08bRAut0QuVScU0y/pnMA4svLwHw4S6O3eNLgk8wi7coQuXVyw3iN+w4igkMT93I+0
g4q59wULArgxYmSionT7pgYyDK6rR/WLsiS7YGLoDDAlzOxOZ4z/4TD7vHZGFD35/iv53HP+5AWlWnBr75FGrTXgFCWmYii8Q==
Vault Nonce (Base64 IV)      : z4LxFE6xVmWGDYvN

[INFO] Memeriksa isi database lokal SQLite Server secara langsung...

--- Record Data Asli di Dalam File Database (Disk) ---
Kolom 'server_share' : eyJpbmRleCI6IDIsICJzaGFyZSI6ICJWR3NvZ0tCcjE2YkJSbGtBZFc3d1BnPT0ifQ==
Kolom 'encrypted_vault': b'\x83\xed\x1a\x8b\xa1\x94\x9511K\x87sA\x7fJ\xef=e\x01\x97'... (Biner Terenkripsi/BLOB)
Kolom 'vault_nonce'   : cf82f1144eb1566586758bcd (Hex Nonce)

[BERHASIL] Konfirmasi: Isi data vault berupa biner acak mentah (BLOB).
[BERHASIL] Konfirmasi: Server TIDAK menyimpan master_key, local_share, ataupun plaintext isi vault.

C:\Usere-Salt\m_salt_school\ITB_STEI-K\II4021 K01 Kriptografi\Tugas 4\PasswordManager>
```

Gambar 3.29 Hasil Pengujian Test Script

Uji Mode Backup

Untuk memastikan bahwa kode mengakses vault backup, server perlu dimatikan.

```
[SERVER INFO] Client must now proceed to sync and save local backup configurations.
127.0.0.1 - - [07/Jun/2026 15:05:51] "PUT /vault/login@mail.com HTTP/1.1" 200 -
[SERVER STATUS] Authentication fetch requested. Retrieving remote cryptographic shares for user 'login@mail.com'...
[SERVER SUCCESS] Dispatched server share and vault payload successfully for user 'login@mail.com'. Client can now process decryption.
127.0.0.1 - - [07/Jun/2026 15:27:52] "GET /vault/login@mail.com HTTP/1.1" 200 -

C:\Users-Salt\m_salt_school\ITB_STEI-K\II4021 K01 Kriptografi\Tugas 4\PasswordManager>
C:\Users-Salt\m_salt_school\ITB_STEI-K\II4021 K01 Kriptografi\Tugas 4\PasswordManager>
C:\Users-Salt\m_salt_school\ITB_STEI-K\II4021 K01 Kriptografi\Tugas 4\PasswordManager>
```

Gambar 3.30 Server Dimatikan

Sistem membuka vault dengan menggunakan master password dan recovery share.

```
===== Password Manager =====
1) Create vault
2) Login (Normal)
3) Login (Backup)
4) Quit
Choose [1/2/3/4] (2): 3
Username: login@mail.com
Master password:
Paste the recovery share string (single line):
Recovery share: ey2pbmrleCI6IDMaIC3:agFyZSI6ICI3Sk41QUU1Y1ROVW9hZmZmNmRmIncGt3PT01fQ==
Restoring vault from backup...
Decrypting local share...
Local share decrypted successfully.
Combining shares via Shamir Secret Sharing...
/ Successfully authenticated as login@mail.com (Read-Only Backup Mode).
/ Logged in as 'login@mail.com' (backup / read-only).

===== Vault Menu ===== (READ-ONLY backup mode)
1) list entries
2) Reveal entry password
6) Logout
Choose [1/2/6] (1):
Vault Entries
=====
# | Service | Username | Password | Notes |
--|---|---|---|---|
1 | github | login00 | ***** | LGONNNA |
2 | guthub | login0 | ***** | fd safe |
=====

===== Vault Menu ===== (READ-ONLY backup mode)
1) list entries
2) Reveal entry password
6) Logout
Choose [1/2/6] (1): 4
Please select one of the available options
Choose [1/2/6] (1): 3
Please select one of the available options
Choose [1/2/6] (1):
```

Gambar 3.31 Backup vault dibuka

Kombinasi kunci dipakai dalam proses pembukaan backup vault

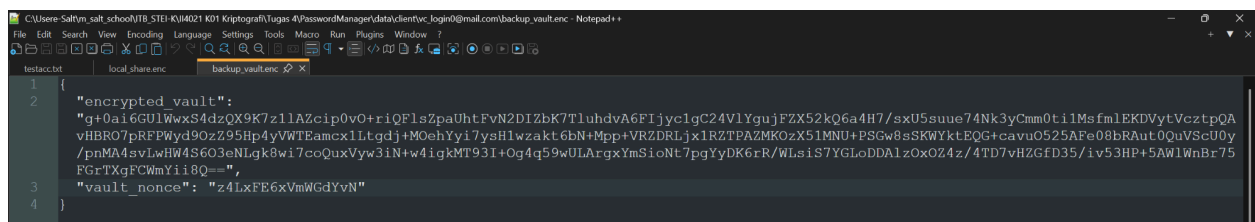

```

Recovery share: eyJpbmR1eCI6IDMsICJzagFyZS16IC13Sk4IQ001Y1ROVXNhZHNkdmlncGE3PT01fQ==
Restoring vault from backup...
Decrypting local share...
✓ Local share decrypted successfully.
Combining shares via Shamir Secret Sharing...
✓ Successfully authenticated as login0@mail.com (Read-Only Backup Mode).
✓ Logged in as 'login0@mail.com' (backup / read-only).

```

Gambar 3.32 Kombinasi Local Share dengan Backup Share

Sistem menggunakan backup vault lokal terenkripsi sebagai sumber vault.



```

1 {
2   "encrypted_vault":
3     "g+0ai6GUlWwxS4dzQX9K7z11AZcip0vO+riQFlsZpaUhtFvN2DIZbK7TluhdvA6FIjyc1gC24V1YgujFZX52kQ6a4H7/sxU5suue74Nk3yCmm0ti1MsfmLEKDVytVcztpQA
4     vHBRO7pRFPWyd90zZ95Hp4yVWTEamcx1Ltgdj+MoehYyi7ysHlwzakt6bN+Mpp+VRZDRLjx1RZTPAZMKOzX51MNU+PSGw8sSKWYktEQG+cavuO525AFe08bRAut0QuVScU0y
      /pnMA4svLWHW4S6O3eNLgk8wi7coQuxVyw3iN+w4igkMT93I+Og4q59wULArgxYmSioNt7pgYyDK6rR/WLsiS7YGLoDDAlzOxOZ4z/4TD7vHZGfD35/iv53HP+5AWlWnBr75
      FGRTXgFCWmYii8Q==",
5   "vault_nonce": "z4LxPE6xVmWGdYvN"
6 }

```

Gambar 3.33 Backup Vault yang diakses

Data password dapat dilihat pada mode backup.



```

Choose [1/2/6] (1): 1
Vault Entries
┌───┬───┬───┬───┬───┐
│ # │ Service │ Username │ Password │ Notes │
├───┴───┴───┴───┴───┘
1 │ github │ login00 │ ..... │ LGONNNA │
2 │ github │ login0  │ ..... │ fd safe │
└───┴───┴───┴───┴───┘

== Vault Menu == (READ-ONLY backup mode)
1) List entries
2) Reveal entry password
6) Logout
Choose [1/2/6] (1): 2
1) github - login00
2) github - login0
0) Cancel
Select entry [0/1/2] (0): 1
Service: github
Username: login00
Password: lgojikoko
Notes: LGONNNA

Apakah Anda ingin menyalin password ini ke clipboard? [y/n] (n): y
Password berhasil disalin ke clipboard!

== Vault Menu == (READ-ONLY backup mode)
1) List entries
2) Reveal entry password
6) Logout
Choose [1/2/6] (1):

```

Gambar 3.34 Data Password diperlihatkan pada Mode Backup

Pengguna tidak dapat menambah, mengubah, atau menghapus data password pada mode backup karena opsi tersebut tidak ada pada mode backup

```
Choose [1/2/6] (1): 1
Vault Entries
```

#	Service	Username	Password	Notes
1	github	login00		LGONNNA
2	guthub	login0		fd safe

```
== Vault Menu == (READ-ONLY backup mode)
1) List entries
2) Reveal entry password
6) Logout
Choose [1/2/6] (1): 3
Please select one of the available options
Choose [1/2/6] (1): 4
Please select one of the available options
Choose [1/2/6] (1): 5
Please select one of the available options
Choose [1/2/6] (1): _
```

Gambar 3.35 Opsi menambah, mengubah, atau menghapus data tidak tersedia

Uji Kegagalan Pemulihan.

Untuk pengujian ini diinputkan recovery share yang salah. Sistem akan gagal merekonstruksi master key atau dekripsi vault gagal.

```

== Password Manager ==
1) Create vault
2) Login (Normal)
3) Login (Backup)
4) Quit
Choose [1/2/3/4] (2): 3
Username: login0@mail.com
Master password:
Paste the recovery share string (single line):
Recovery share: eyJpbmRleCI6IDMsICJzaGFyZSI6ICI3Sk41QUU1Y1ROVXNnZHNkdm1nssssssssss==
Restoring vault from backup...
Decrypting local share...
✓ Local share decrypted successfully.
Error: Format string recovery share tidak valid.
Error: Backup login failed. Check password and recovery share.

== Password Manager ==
1) Create vault
2) Login (Normal)
3) Login (Backup)
4) Quit
Choose [1/2/3/4] (2):

```

Gambar 3.36 Dekripsi Backup Vault Gagal karena Recovery Share yang Salah

Pengujian selanjutnya akan menggunakan backup vault yang tidak sesuai atau sudah dimodifikasi, dekripsi AES-128-GCM gagal karena validasi autentikasi tidak sesuai.

```

1 {
2   "encrypted_vault":
3     "g+0a16GulWwxS4dzQX9K7z11AZcip0vO+r1QFlsZpaUhtFvN2DIzbK7T1uhdvA6FIjyc1gC24V1YgujFZX52k06a4H7/sxU5uuue74Nk3yCmm0t11Msfm1EKDVytVcztpQAvHBR07pRFPWyd9OzZ95Hp4yVWTEamcx1Ltgdj+MOehYyi7ysH1wzakt6bN+Mpp+VRZDRljx1RZTPAZMKOzX51MNU+PSGw8aSKWYktEQG+cavuo525AFe08bRAut0QuVSc00y/pnMA4svlwHw4S603eNLgk8w17coQuXVyw3iN+w4igkMT93I+Og4q59wULArgxYmSioNt7pgYyDK6rR/WLsiS7YGLoDDA1z0xOZ4z/4TD7vHZGFd35/iv53HP+5AW1WnBr75FGrTXgPCWmYi18Q==",
4   "vault_nonce": "z4LxFE6xVmWGdYvN"
}

```

Gambar 3.37 Kondisi Awal Vault

```

1 {
2   "encrypted_vault":
3     "g+0a16GulWwxS4dzQX9K7z11AZcip0vO+r1QFlsZpaUhtFvN2DIzbK7T1uhdvA6FIjyc1gC24V1YgujFZX52k06a4H7/sxU5uuue74Nk3yCmm0t11Msfm1EKDVytVcztpQAvHBR07pRFPWyd9OzZ95Hp4yVWTEamcx1Ltgdj+MOehYyi7ysH1wzakt6bN+Mpp+VRZDRljx1RZTPAZMKOzX51MNU+PSGw8aSKWYktEQG+cavuo525AFe08bRAut0QuVSc00y/pnMA4svlwHw4S603eNLgk8w17coQuXVyw3iN+w4igkMT93I+Og4q59wULArgxYmSioNt7pgYyDK6rR/WLsiS7YGLoDDA1z0xOZ4z/4TD7vHZGFd35/iv53HP+5AW1WnBr75FGrTXgPCWmYi18Q==",
4   "vault_nonce": "z4LxFE6xVmWGdYvNv"
}

```

Gambar 3.38 Kondisi Vault Dengan Nonce Diubah

Pada pengujian ini nonce diubah menjadi dari “z4LxFE6xVmWGdYvN” menjadi “z4LxFE6xVmWGdYvNv”.

```
== Password Manager ==
1) Create vault
2) Login (Normal)
3) Login (Backup)
4) Quit
Choose [1/2/3/4] (2): 3
Username: login@mail.com
Master password:
Paste the recovery share string (single line):
Recovery share: eyJpbmRleCI6IDMsICJzaGFyZSI6ICI3Sk41QUU1Y1ROVXNnZHNkdmlncGt3PT0ifQ==
Restoring vault from backup...
Decrypting local share...
✓ Local share decrypted successfully.
Combining shares via Shamir Secret Sharing...
Error: Backup Login Failed: Gagal mendekripsi file cadangan vault menggunakan master key hasil gabungan SSS. ()
Error: Backup login failed. Check password and recovery share.

== Password Manager ==
1) Create vault
2) Login (Normal)
3) Login (Backup)
4) Quit
Choose [1/2/3/4] (2): _
```

Gambar 3.39 Vault Backup Tidak dapat diakses

Dari gambar diatas, menunjukan bahwa sistem tidak menampilkan data password ketika proses dekripsi gagal. Data password tidak ditampilkan ketika proses dekripsi gagal.

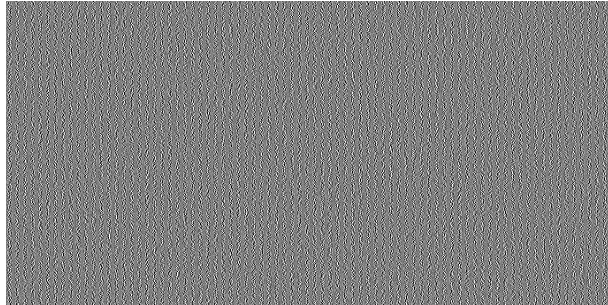
Uji Kriptografi Visual *bonus*

Ditampilkan QR code awal dari recovery share.

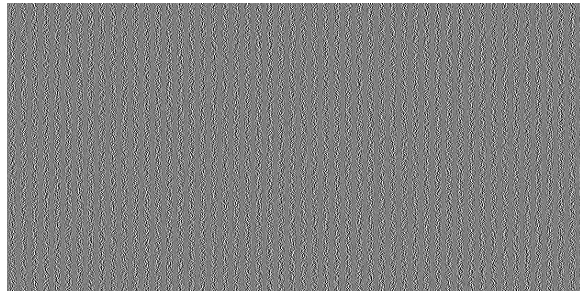


Gambar 3.40 QR code Recovery Share Awal

Berikut adalah dua gambar share hasil pembagian QR code.

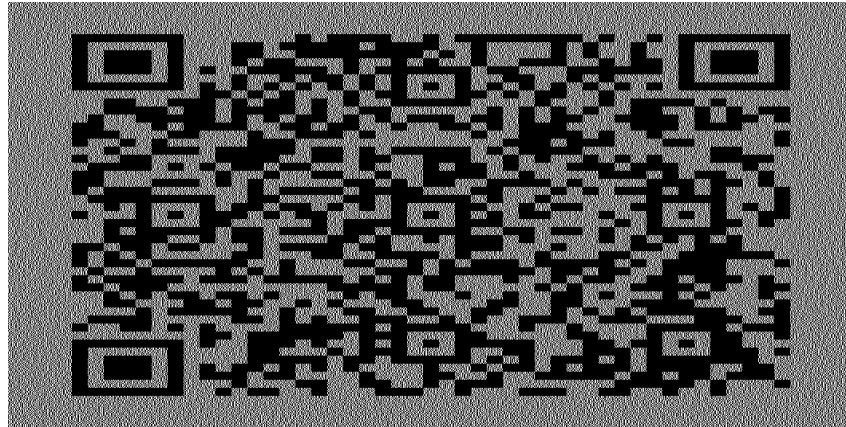


Gambar 3.41 Share 1



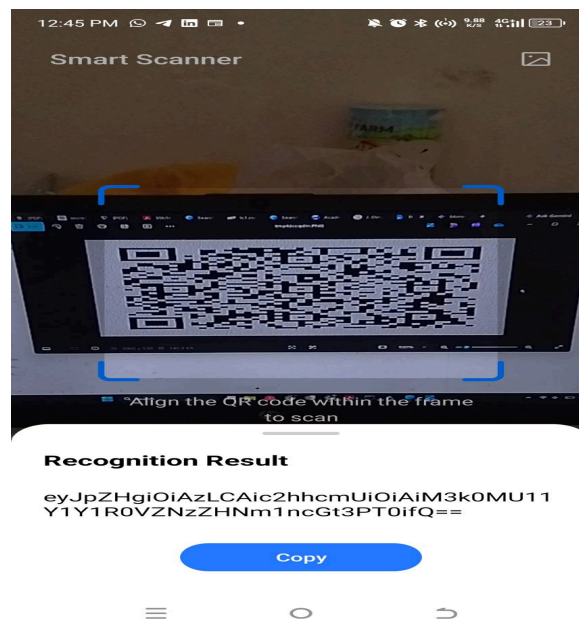
Gambar 3.42 Share 2

Hasil penggabungan kedua gambar share. Dalam pengujian ini digunakan script python. Untuk menggabungkannya



Gambar 3.43 Hasil penggabungan share

QR code hasil penggabungan dapat dipindai dan menghasilkan recovery share yang sama.



Gambar 3.44 Hasil Scan QR rekonstruksi

Bukti hasil decode QR sama dengan recovery share akan memakai string checker untuk membandingkannya.

```
recovery_text = "eyJpZHgiOiAzLCAic2hhcmUiOiAiM3k0MU11Y1Y1R0VZNzZHNm1ncGt3PT0ifQ=="
if recovery_text == decode_qr_image(hasil_tumpuk):
    print("Berhasil! Hasil scan sesuai dengan recovery share awal.")
```

Gambar 3.45 Script String checker

```
Berhasil! Hasil scan sesuai dengan recovery share awal.
```

Gambar 3.46 Hasil Pengecekan String

Dari hasil tersebut, bahwa QR berhasil direkonstruksi dari dua image share tersebut.

Kesimpulan dari Hasil Implementasi

Berdasarkan hasil implementasi yang telah dilakukan, dapat disimpulkan bahwa sistem Password Manager terdistribusi ini telah berhasil mengintegrasikan protokol kriptografi untuk menjamin kerahasiaan, integritas, dan ketersediaan data pengguna secara optimal. Melalui kombinasi algoritma AES-GCM untuk enkripsi data utama dan fungsi derivasi kunci Argon2id, sistem mampu menangkal serangan brute-force sekaligus memastikan autentikasi data yang kuat. Inti dari keamanan arsitektur terdistribusi ini terletak pada penerapan metode Shamir's Secret Sharing di

mana kunci enkripsi dipecah menjadi 3 *shares* (*recovery share*, *local share*, dan *server share*) yang masing-masing disimpan di tempat yang berbeda. Data yang tersimpan dalam *vault* hanya dapat diakses jika jumlah *shares* yang dikumpulkan memenuhi *threshold* minimum yang telah ditentukan (2 dari 3).

Selain itu, implementasi kriptografi visual berbasis skema (2,2) juga terbukti efektif dalam mengamankan kode QR autentikasi di mana kode tersebut dipecah menjadi dua citra acak yang tidak bermakna dan hanya dapat membentuk sebuah QR yang dapat di-*scan* untuk mendapatkan *recovery share* ketika kedua citra tersebut digabung baik secara digital maupun secara fisik. Sistem ini berhasil mewujudkan konsep Zero-Knowledge, di mana penyedia layanan atau pihak ketiga tidak memiliki akses sama sekali terhadap informasi asli milik pengguna sehingga memberikan perlindungan data yang aman, terdistribusi, dan *fault-tolerant*.

Daftar Pustaka

- **Yakkala, P.** (2023). *Understanding AES Encryption and AES GCM Mode: An In-Depth Exploration Using Java*. Medium. URL: <https://medium.com/@pravallikayakkala123/understanding-aes-encryption-and-aes-gcm-mode-an-in-depth-exploration-using-java-e03be85a3faa>
- **GeeksforGeeks.** (2023). *Implementing Shamir's Secret Sharing Scheme in Python*. GeeksforGeeks. URL: <https://www.geeksforgeeks.org/implementing-shamirs-secret-sharing-scheme-in-python/>
- **Buchanan, W. J.** (2023). *Argon2*. Medium - A Security Site. URL: <https://medium.com/asecuritysite-when-bob-met-alice/argon2-ffb941a548c6>
- **Catatan Kriptografi.** (2018). *Program Kriptografi Visual untuk Skema (2, 2)*. WordPress. URL: <https://catatankriptografi.wordpress.com/2018/02/20/program-kriptografi-visual-untuk-skema-2-2/>

Lampiran

Link Repositori : <https://github.com/echaa0018/PasswordManager>

Link Video Demo : <https://youtu.be/9imeGRV7rIE>

Pembagian Tugas :

No	Anggota - NIM	Tugas
1	Raja M Arkan - 10123065	1. Laporan: a. Teori Singkat b. Uji Coba Program
2	Muhammad Arya Putra Prihastono - 18223068	1. Source Code: a. Bug Fixing b. Additional Logging 2. Laporan a. Uji Coba Program
3	Mahesa Satria Prayata - 18223082	1. Source code 2. Laporan: a. Teori Singkat b. Perancangan dan Implementasi